

2. Разработка архитектуры системы

В данном разделе описывается архитектура мобильного приложения «Гостиница» после реализации практических этапов 3-6, версия 0.6.0. Проект разработан на Kotlin для Android в соответствии с индивидуальным заданием Арзимуротова А.М., группа ПИН-123. Номера практических этапов не являются номерами глав пояснительной записки. В итоговой записке архитектурные решения и алгоритмы включаются в предусмотренную заданием главу «Разработка алгоритмов» с согласованием нумерации.

Реализованы SQL-каталог, учетные записи, поиск свободных номеров, транзакционное бронирование, демонстрационная оплата, история, отмена и PDF-подтверждение. Система содержит три государства, шесть городов, 36 вымышленных гостиниц категории 3-5 звезд, 108 типов и 432 физических номера, а также 30 информационных мест. Рестораны и объекты отдыха не бронируются и не продают билеты. Фотографии являются встроенными учебными иллюстрациями. Реальные банковские платежи, вызов автомобиля и передача заказа внешним компаниям отсутствуют.

2.1 Выбор архитектурного подхода

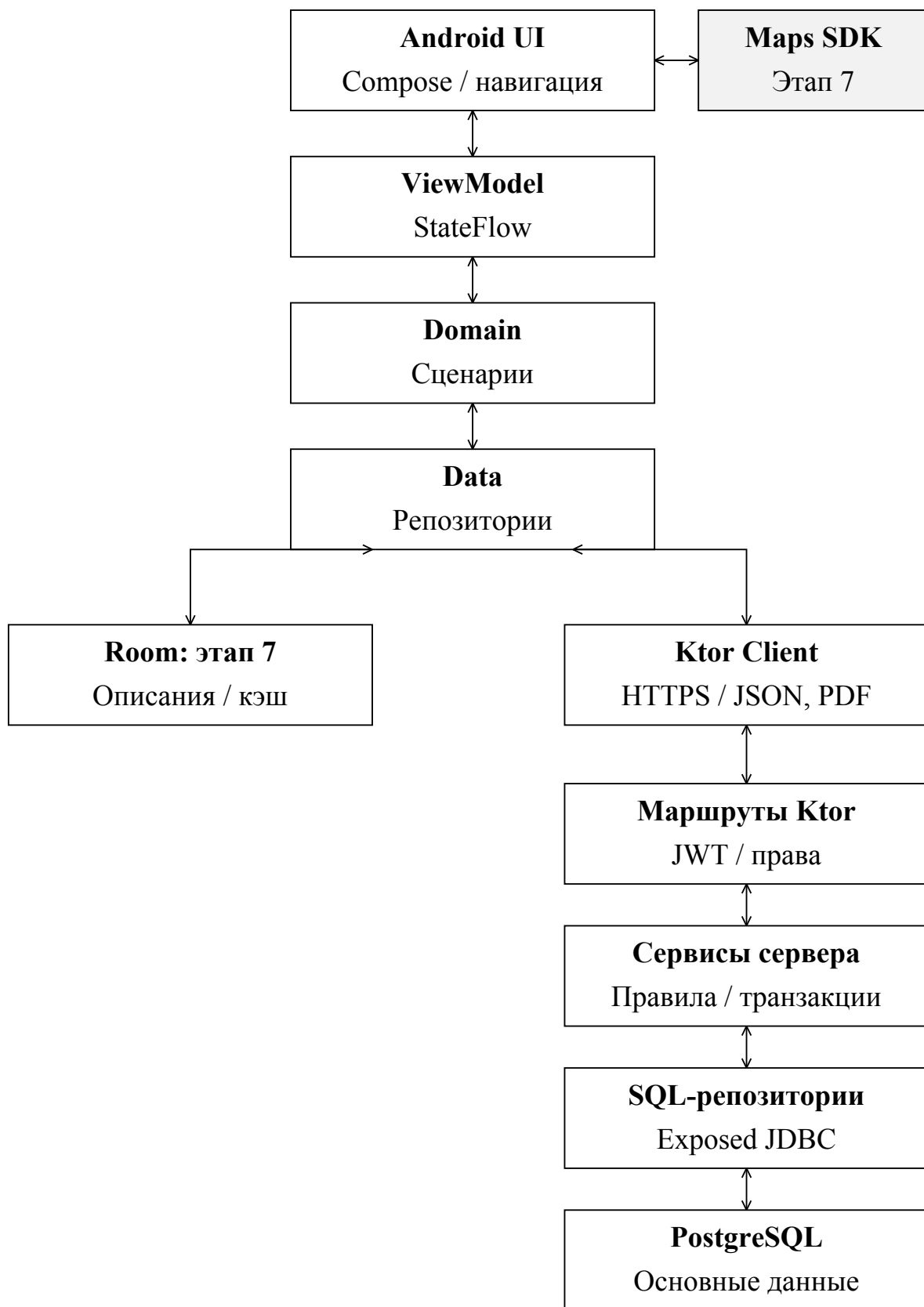
Выбрана трехуровневая клиент-серверная архитектура: Android-приложение, Ktor REST API и PostgreSQL. Android отвечает за представление данных и пользовательские действия; сервер проверяет права, доступность, сроки и стоимость; PostgreSQL обеспечивает долговременное хранение и транзакционную согласованность. Клиент не подключается к SQL напрямую и не получает пароли базы.

Сервер представляет собой модульный монолит. Учетные записи, каталог, поиск, заказы, демооплата и документы разделены логически, но выполняются в одном процессе. Такой подход позволяет фиксировать заказ, услуги и выделение комнат одной транзакцией без распределенных протоколов. Использование Kotlin на обеих сторонах допускает общий исходный каталог DTO, однако UI и SQL-реализации остаются независимыми.

Код хранится в GitHub. Репозиторий содержит независимые Gradle-проекты android и server, общий каталог shared, документацию docs,

версии зависимостей gradle и переносимые скрипты scripts. Gradle Wrapper фиксирует среду сборки. Android Studio открывает каталог android из полного репозитория: shared и gradle не отделяются от него. На Mac и Asus используются независимые локальные PostgreSQL, учетные записи и секреты; Git синхронизирует код, а не SQL-данные.

Рисунок 2.1 - Архитектура компонентов; Room и Maps обозначены как будущий этап 7



2.2 Архитектура Android-приложения

Интерфейс выполнен на Jetpack Compose и Material 3. Используются русские строки, единый стиль карточек, галерея, нижняя навигация на телефоне и боковая панель на широком экране. Главные разделы: «Поиск», «Места», «Брони», «Профиль». Предусмотрены загрузка, ошибка, пустой результат, повтор и состояние выполняющейся операции.

Применяется MVVM и разделение UI - Domain - Data. Compose получает неизменяемые состояния и callbacks; ViewModel управляет сценариями; репозитории обращаются к HTTP. Hilt связывает интерфейсы с реализациями. StateFlow наблюдается с учетом жизненного цикла Activity. Coroutines позволяют выполнять сетевые операции без блокирования главного потока. Выбран Ktor Client; одновременное применение Retrofit не требуется.

Таблица 2.1 - Ответственность клиентских компонентов

Компонент	Назначение
BrowseViewModel	Каталог, выбранное направление, даты, гости, фильтры и избранное. SavedStateHandle сохраняет совместимые с Bundle значения формы.
AuthViewModel	Режим гостя, регистрация, вход, профиль, выход и сообщения об истечении сеанса.
TravelViewModel	Независимые jobs поиска и наличия; последовательные операции заказа, оплаты, отмены и получения PDF.
FoundationViewModel	Диагностика реального соединения Android - Ktor - PostgreSQL.
API-каталог	RemoteCatalogRepository загружает SQL-каталог; скрытого возврата к локальным demo-данным нет.

Компонент	Назначение
API-заказы	RemoteTravelRepository выполняет поиск, оформление, демооплату, отмену и получение документа.
SessionStore	JWT и ключ незавершенного POST шифруются AES-GCM ключом Android Keystore в noBackupFilesDir.

Поиск результатов выполняется на сервере, а не локальной функцией фильтрации каталога. Предыдущий запрос отменяется при изменении формы; ввод названия имеет задержку 300 мс. Запоздавший ответ не заменяет результаты нового запроса. Список загружается по 20 гостиниц. Сохраненные в RAM описания не обозначаются как актуальное наличие. Постоянный offline-кеш Room остается этапом 7.

Карточка гостиницы повторно запрашивает наличие. Экран оформления использует roomTypeId и rateId серверного предложения и актуальные цены услуг из ответа availability. После изменения параметров старая выбранная цена не используется для новой брони. Главный экран и избранное являются обзором каталога; цена каталога не гарантирует свободную комнату на даты.

Пароль не сохраняется в Bundle, файлах или журналах. JWT удаляется при выходе и защищенном ответе 401. Сетевой сбой не считается успешной оплатой. Ключ незавершенного создания/оплаты хранится зашифрованно и повторяется при потере ответа. Отдельная блокировка кнопки улучшает UX, но защиту от дублей обеспечивает серверная идемпотентность.

2.3 Архитектура серверной части и REST API

Сервер организован как routes - services - repositories. HTTP-слой разбирает DTO и параметры; сервисы задают правила; SQL-репозитории выполняют prepared-запросы в транзакциях Exposed. HikariCP ограничивает пул четырьмя соединениями. JDBC и Argon2 выполняются на Dispatchers.IO, а не в Netty event loop. Фиктивный успех при недоступной SQL-базе не возвращается.

Публичная регистрация создает только USER. ADMIN создается отдельной локальной командой, без общего стандартного пароля. Пароли хешируются Argon2id: индивидуальный salt, 19 MiB памяти, две итерации и параллелизм 1. JWT действует 30 минут; проверяются подпись, issuer, audience, срок и UUID пользователя. Роль и активность читаются из SQL при каждом защищенном запросе. Refresh и восстановление пароля в данной версии не реализованы.

Общий формат ошибки содержит code, message, fieldErrors и requestId. Внутренние SQL, пароль, token и stack trace клиенту не передаются. Неправильные поля дают 400, отсутствующая авторизация - 401, запрещенная роль - 403, отсутствующий или чужой заказ - 404, конфликт цены/наличия/состояния/ключа - 409. Регистрация и вход ограничены по частоте; приложение не доверяет пользовательским полям role и active.

Таблица 2.2 - Основные API-группы, префикс /api/v1

Маршруты	Назначение и права
/auth/register; /auth/login	Регистрация и JWT-вход по email/паролю.
/profile GET/PATCH	Только собственный профиль; редактирование имени и телефона.
/locations; /hotels; /rooms; /places	Публичные справочные данные; /rooms не гарантирует наличие.
/hotels/search	Серверный SQL-поиск свободных комнат на весь диапазон с фильтрами, сортировкой и пагинацией.
/hotels/{id}/availability	Актуальные типы/тарифы/остаток комнат и цены дополнительных услуг.
/bookings POST/GET; /bookings/{id}	Создание, список и подробности только собственных заказов.
/bookings/{id}/cancel	Отмена собственного заказа по сохраненной политике.

Маршруты	Назначение и права
/payments/demo POST	Подтверждение действующего собственного ONLINE_DEMO резерва; реквизиты карты отсутствуют.
/receipts/{id} GET	Нефискальный PDF только владельцу, application/pdf и private no-store.
/admin/summary	Read-only SQL-сводка для активного ADMIN; полный CRUD остается этапом 8.

2.3.1 Алгоритм поиска

Запрос задает государство, необязательный город, даты, взрослых, детей и количество комнат. Интервал проживания имеет вид [checkIn, checkOut): день выезда не добавляет ночь. Допускаются 1-90 ночей, 1-4 комнаты одного типа, 1-8 взрослых и 0-4 ребенка. На каждую комнату нужен взрослый; дети учитываются при проверке вместимости без возрастных тарифов. Одна страна задает валюту; автоматическая конвертация не выполняется.

SQL отбирает активные географические и гостиничные записи, физические комнаты нужного типа и тариф, покрывающий весь период. Удобства должны присутствовать у того же типа, который соответствует гостям и бюджету. Не допускается суммировать свободные комнаты разных типов либо объединять комнаты, свободные в разные части поездки, в одну доступную комнату.

Комната исключается при пересечении с активной room_blocks, подтвержденной/завершенной бронью либо PENDING_PAYMENT с expiresAt позднее проверяемого момента. Пересечение определяется условием $startA < endB$ и $endA > startB$. Соседние интервалы без общей ночи допускаются. Цена за весь период равна ночной цене, умноженной на количество ночей и комнат; услуги в результат поиска не включены.

Фильтры охватывают звезды 3-5, рейтинг, цену, тип, удобства, расстояние и буквальное вхождение названия. SQL-значения

параметризованы; пользовательский текст не становится условием запроса. Сортировка задается доверенным перечислением: рекомендации, цена, рейтинг или расстояние; UUID стабилизирует порядок при равенстве. Limit и offset ограничивают гостиницы в SQL. Счетчик и страница читаются согласованно в REPEATABLE READ.

Учебные room_blocks демонстрируют влияние дат без подмены их клиентскими бронями. Первая дата заполнения сохраняется в app_metadata, закрытый период начинается через семь дней и длится три дня. Повторное заполнение не переносит период и не затирает изменения. После реализации бронирования результаты дополнительно изменяются собственными SQL-заказами.

2.3.2 Алгоритм создания заказа

Проверка наличия на экране недостаточна: два пользователя могут увидеть одну комнату. Сервер проверяет принадлежность типа и тарифа гостинице, активность записей, условия оплаты и дополнительные услуги. В транзакции READ COMMITTED блокируются необходимые каталожные записи, затем все комнаты типа в порядке UUID. Отдельный последующий запрос повторно читает занятость после получения блокировок. Услуги и тарифы также защищены от изменения во время расчета.

Все операции над существующим заказом используют один порядок: комнаты UUID ASC, затем строка заказа и связанный платеж. Поэтому подтверждение оплаты, отмена и обслуживание резервов согласованы с созданием новой брони. При недостатке комнат вся транзакция откатывается: частично выделенные комнаты и услуги не остаются.

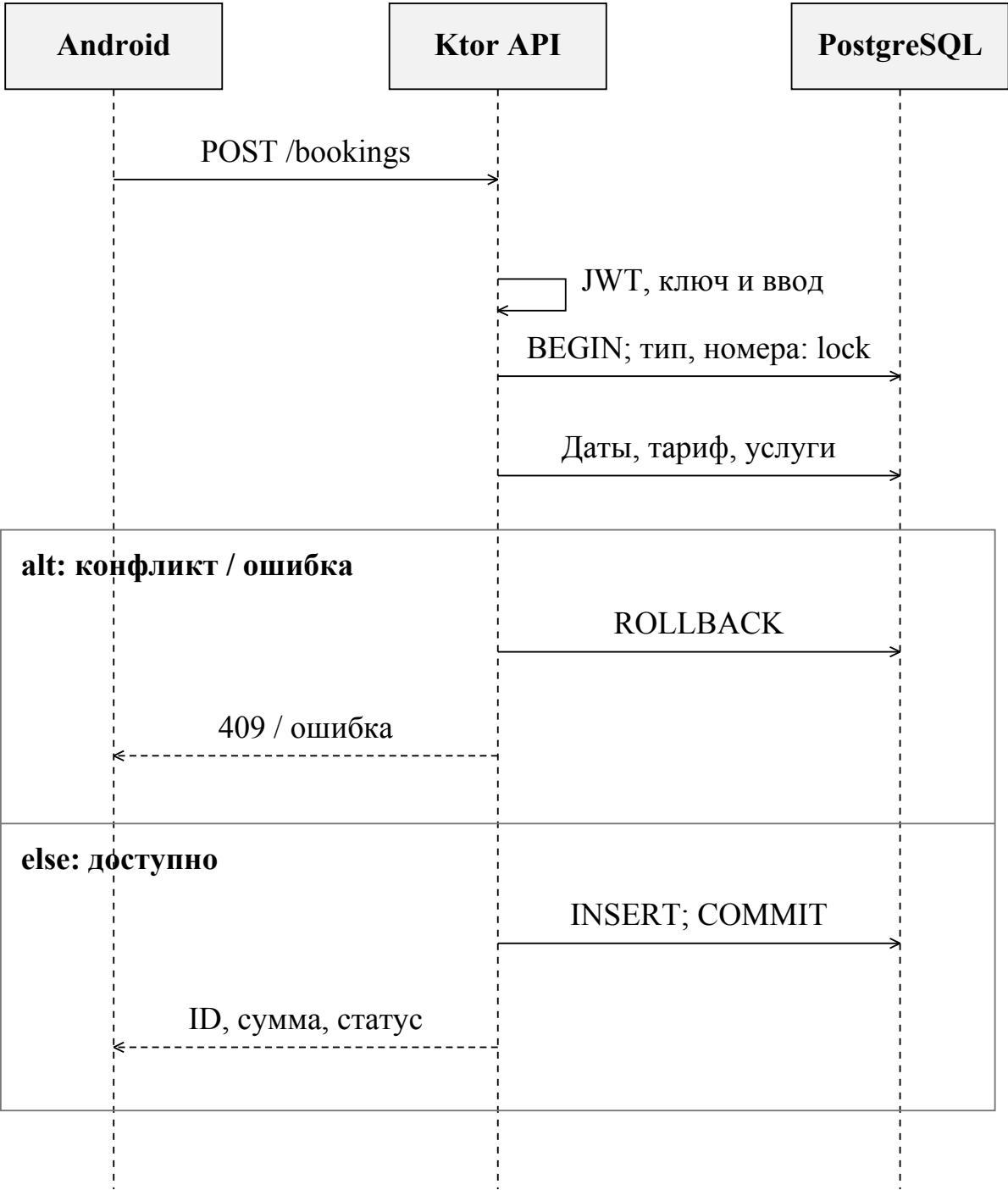
Сервер вычисляет итоговую цену и сравнивает ее с expectedTotal как условием согласия пользователя. Расхождение дает PRICE_CHANGED, а не молчаливое принятие новой суммы. Недоступность дает AVAILABILITY_CHANGED. PER_NIGHT_ROOM услуги начисляются за каждую ночь каждой комнаты; ONCE - один раз на заказ. Для встречи в аэропорту сохраняются аэропорт, рейс, местное время и телефон, но заказ не отправляется перевозчику.

Одному заказу выделяется 1-4 физические комнаты одного типа через booking_rooms. Вместе с заказом сохраняются услуги и неизменяемые

реквизиты клиента/гостиницы/комнат/цен/политики. Позднейшее изменение тарифа или названия не переписывает историю.

Пара `user_id` и `idempotency_key` уникальна. Отпечаток нормализованного запроса позволяет вернуть прежний заказ при повторе, а другое содержимое с тем же ключом отвергается. Advisory lock сериализует совпадающие ключи до выделения комнат. Повтор не создает новый заказ, не продлевает резерв и не восстанавливает отмененную бронь.

Рисунок 2.2 - Последовательность создания брони



ONLINE_DEMO: ожидание оплаты, 15 минут.

PAY_AT_HOTEL: подтверждено, без оплаты.

2.3.3 Оплата, отмена и обслуживание

ONLINE_DEMO создает PENDING_PAYMENT с резервом на 900 секунд. PAY_AT_HOTEL сразу создает CONFIRMED с UNPAID и не создает онлайн-платеж или подтверждение оплаченной суммы. Способ оплаты фиксируется при оформлении. Заезд установлен на 14:00 и выезд на 12:00 в IANA-зоне города гостиницы: Asia/Tashkent, Europe/Moscow или Europe/Istanbul.

Ожидающий резерв можно отменить. Подтвержденный заказ отменяется только до cancelUntil, вычисленного из сохраненной тарифной политики; исходные тарифы задают 24 часа до заезда. После границы частичный возврат/штраф не моделируется, действие отклоняется. При отмене оплаченного заказа статус платежа становится REVERSED: это учебное аннулирование, не банковский возврат.

Демооплата разрешена только владельцу действующего ONLINE_DEMO резерва. После блокировок повторно проверяются статус и срок по серверным часам. В одной транзакции создается уникальный платеж, заказ переводится в CONFIRMED/PAID и сохраняется снимок PDF. UNIQUE(booking_id) допускает не более одного успешного демоплатежа. Повтор прежнего ключа возвращает прежний результат; новый ключ уже оплаченного заказа не создает вторую операцию.

Фоновая coroutine раз в минуту и при запуске сервера переводит просроченные резервы в CANCELLED с EXPIRED, а подтвержденные заказы после checkout_at - в COMPLETED. Корректность поиска не зависит от обслуживания: истекший резерв проверяется непосредственно через expiresAt. GET истекшего заказа отображает недействующий резерв, не производя скрытую запись. Противоположные переходы из CANCELLED/COMPLETED в активное состояние отсутствуют.

2.4 Архитектура базы данных

Изменения схемы выполняются Flyway, без destructive clean и пересоздания таблиц. V1 содержит метаданные; V2 вводит каталог и пользователей; V3 добавляет room_blocks, часовые пояса и запрет пересечения активных тарифов через EXCLUDE и btree_gist; V4 - заказы и их состав; V5 - демоплатежи и документы. Примененные миграции не редактируются. Ошибка миграции останавливает запуск, а не обходится удалением ограничений.

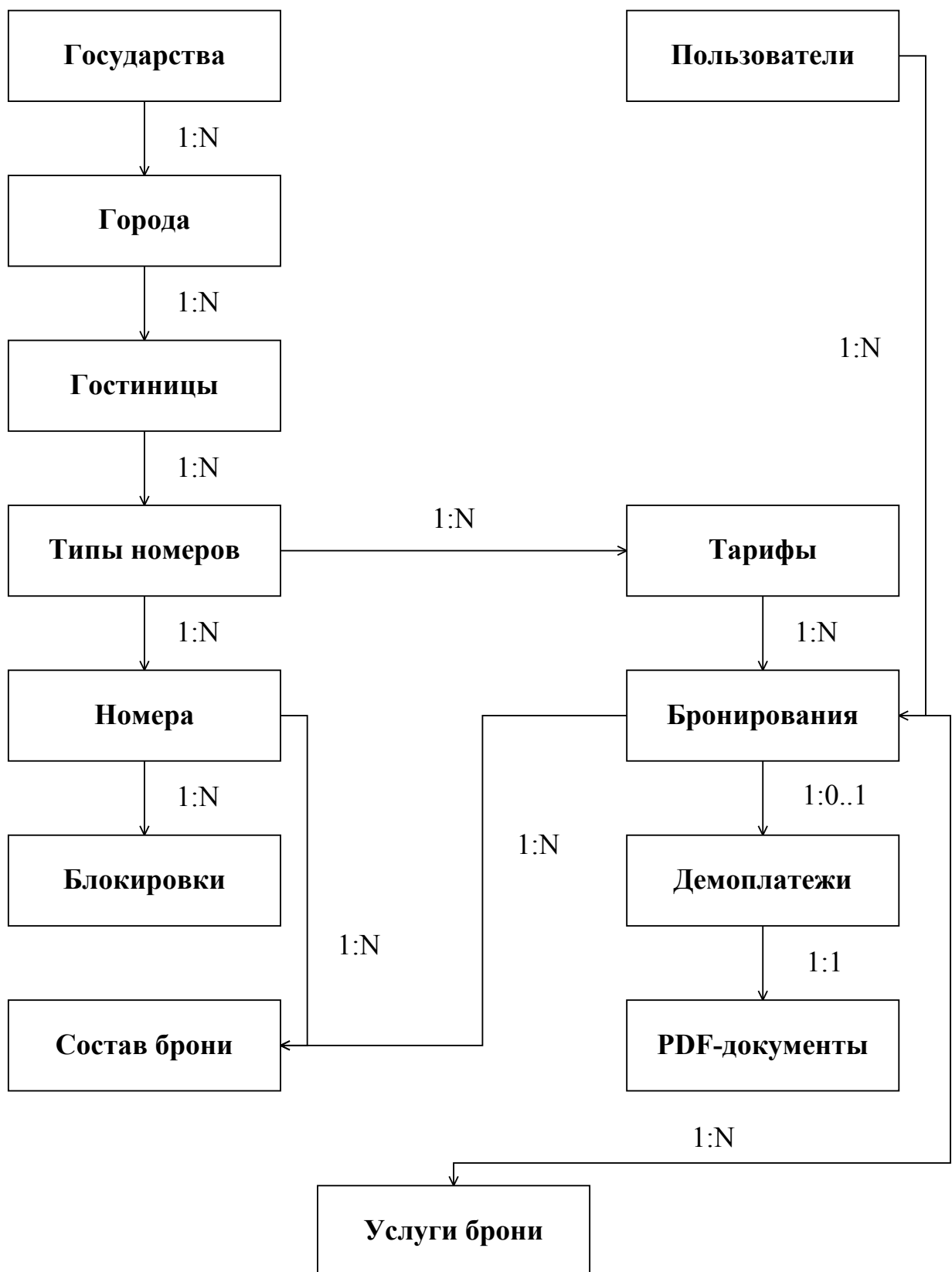
Основные ключи - UUID; даты проживания - DATE; моменты операций - TIMESTAMPTZ. Деньги хранятся как NUMERIC(12,2), в DTO передаются Long в минимальных единицах, в серверном расчете используются BigDecimal и точные целочисленные операции. Float/Double допустимы для рейтинга и расстояния, но не для сумм. Гостиница связана со страной через город; физический номер - с гостиницей через тип.

Таблица 2.3 - Состав реализованной SQL-модели

Таблицы	Назначение и ограничения
users	Нормализованный UNIQUE email, hash, имя/телефон, role USER/ADMIN, active.
countries, cities, hotels	География, валюта страны, IANA-зона города, категория/рейтинг/адрес гостиницы, active и устойчивый catalog_key.
room_types, rooms, rates	Вместимость/площадь/тип, физические комнаты, тарифы и условия оплаты/отмены. Непересекающиеся активные интервалы тарифа.
amenities и room_type_amenities	Нормализованные удобства и связь с конкретным типом номера.
services и hotel_services	Вид услуги, стоимость предложения гостиницы, PER_NIGHT_ROOM или ONCE.

Таблицы	Назначение и ограничения
hotel_photos, room_type_photos	Ключи встроенных иллюстраций и порядок показа.
places и hotel_places	Общие городские места и M:N-рекомендации гостиниц без дублирования ресторана.
room_blocks	Периоды закрытия физической комнаты; учебные записи не являются заказами.
bookings и booking_rooms	Владелец, тип/тариф, период, состояния, цена, сроки, отпечаток/ключ и состав из физических комнат.
booking_services	Выбранные услуги с сохраненной суммой и реквизитами трансфера.
payments и receipts	Один успешный демоплатеж на заказ, уникальная транзакция и воспроизводимый снимок документа.

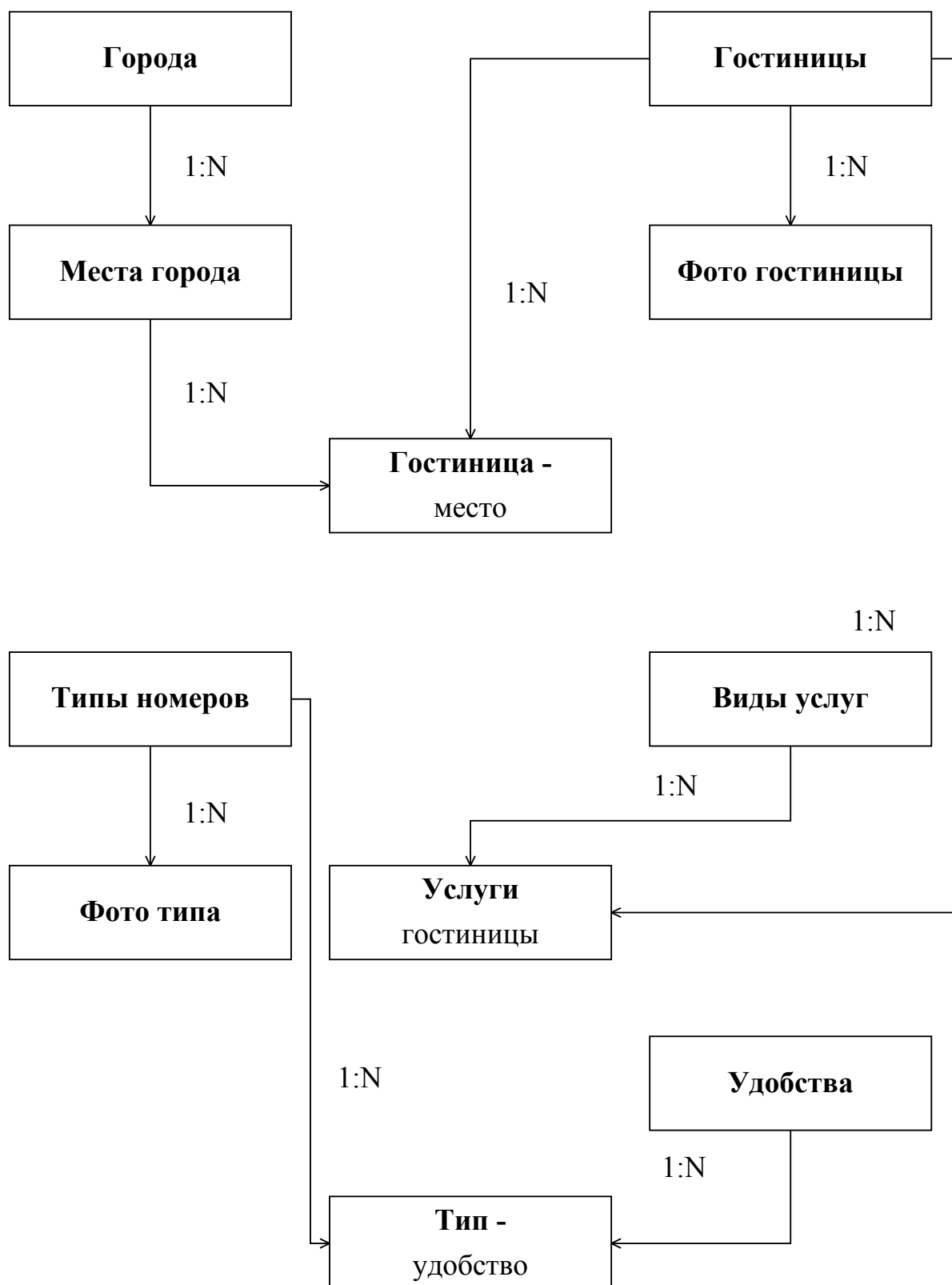
Рисунок 2.3 - Основные связи SQL; «Состав брони» означает booking_rooms



Один заказ содержит несколько строк booking_rooms, каждая ссылается на физическую комнату. Это уточняет ранний проект с одной room_id непосредственно в bookings и соответствует форме выбора нескольких

комнат. JSONB details_snapshot и document_snapshot дополняют внешние ключи, но не заменяют числовые поля и индексы для поиска.

Рисунок 2.4 - Связи дополнительного каталога



Исходный seed: 3 countries, 6 cities, 36 hotels, 108 room_types, 432 rooms, 108 rates, 7 amenities, 3 services, 108 hotel_services, 30 places и 180 hotel_places. Устойчивые UUID и ON CONFLICT DO NOTHING предотвращают дубли и не затирают правки. История не удаляется каскадно. Полноценное редактирование и дополнительные правила администратора остаются этапом 8.

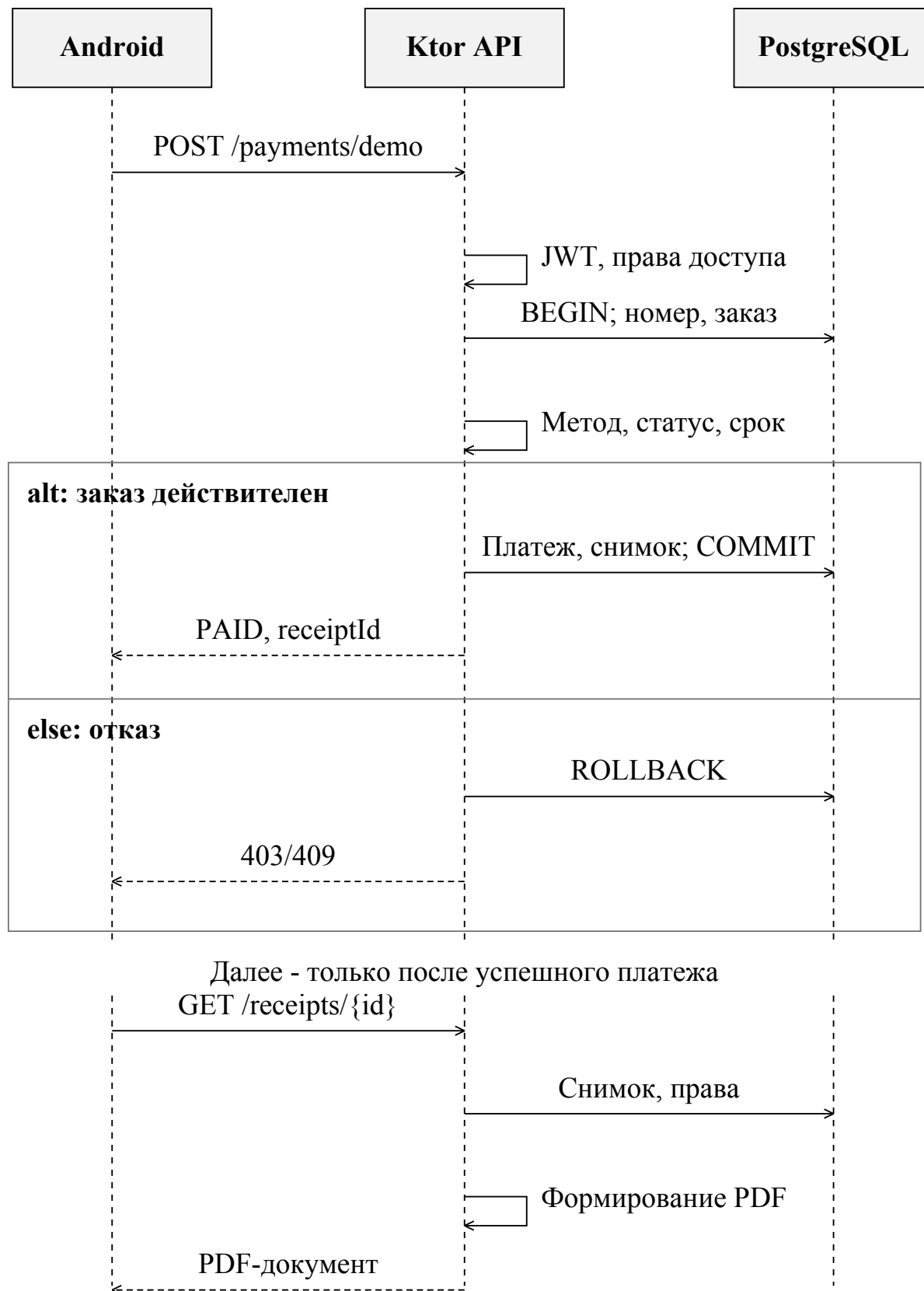
2.5 Формирование PDF и взаимодействие компонентов

После фиксации успешного платежа ReceiptService получает сохраненный снимок и формирует PDF посредством Apache PDFBox 3.0.8. Шрифт DejaVu Sans с кириллицей включен в ресурсы сервера вместе с лицензией; системные шрифты Mac или Windows не требуются. Предусмотрены перенос длинных строк и переход на следующую страницу. Символы вне набора шрифта безопасно заменяются, а не повреждают платеж.

Документ содержит номер заказа, клиента, гостиницу/адрес, тип и физические комнаты, даты/гостей, стоимость проживания и услуг, валюту, время и метод оплаты, transactionId и receiptId. Обязательно предупреждение «Демонстрационный документ. Не является фискальным чеком». Сбой генерации после COMMIT не отменяет оплату: повтор GET создает документ по тем же реквизитам. При отмене исходное PAID-подтверждение сохраняется как историческое, а актуальное аннулирование показывается в заказе.

Android сохраняет PDF через Storage Access Framework, без разрешения на все хранилище. Печать передается Android Print Framework и системному диалогу. Наличие программного диалога не означает подтвержденную печать на физическом принтере. Документы доступны только владельцу; выход из аккаунта очищает личные состояния приложения.

Рисунок 2.5 - Демооплата и последующее получение PDF



Полный поток включает поиск, повторное наличие, вход, оформление и выбор оплаты. ONLINE_DEMO показывает серверный таймер; PAY_AT_HOTEL сразу показывает подтвержденный неоплаченный заказ.

Таймер клиента информационный: сервер решает, действителен ли резерв. При потере сети пользователь получает ошибку и может повторить запрос с сохраненным ключом, а не новый скрытый заказ.

2.6 Обеспечение требований и проверка

Таблица 2.4 - Связь задания с реализацией и оставшимися этапами

Требование задания	Обеспечение
Android/Kotlin, каталог и фильтры	Compose, MVVM и SQL-каталог/поиск с проверкой физического номера на весь период.
Бронирование и услуги	Транзакции, блокировки, повторная проверка, booking_rooms/booking_services, снимок цены.
Личный кабинет и история	JWT, профиль, GET собственных заказов, активные/исторические состояния и отмена.
SQL и кабинет администратора	PostgreSQL/Flyway и read-only ADMIN-сводка уже реализованы; добавление/редактирование через кабинет - этап 8.
Dokka и GitHub	Русские KDoc, генерация HTML в обоих проектах, version control и Windows CI.
Модульное и автоматизированное UI-тестирование	ViewModel/репозитории/JWT/health, PostgreSQL интеграционные и Compose сценарии.
Не менее трех виртуальных устройств	Итоговая матрица этапа 9: API 30 компактный телефон, API 35 обычный телефон, API 36 планшет; Xiaomi не заменяет три AVD.
Структуры данных, листинг с комментариями и скриншоты	ER/API и русские пояснения обновлены; итоговые приложения и руководства дополняются на этапе 9.

SQL-интеграционные тесты выполняются только в отдельной базе `hotel_coursework_test` с защищенным именем. Основная база не очищается. Проверяются `fresh/upgrade` миграции, повторный `seed`, сохранение правок, роли/JWT, конкурирующие брони, соседние интервалы, отсутствие агрегирования разных комнат по дням, стоимость услуг, идемпотентность и неизменность истории. Отдельно проверяются гонка оплаты и отмены, истечение резерва, завершение проживания и повтор PDF после искусственного сбоя генератора.

Compose-тесты проверяют регистрацию/профиль, формы/навигацию/фильтры, наличие обязательного входа в `checkout`, серверную цену, выбор оплаты, запрет повторного подтверждения и исторический документ отмененного заказа. API-интеграция Android дополнительно проверяется на Xiaomi 12 Pro через USB. Автоматизированный runner принимает только явно заданный `emulator-NNNN`; личный телефон обновляется `install -г`, без удаления приложения и `clear data`.

Локальный backend слушает только `127.0.0.1:8080`. Debug допускает HTTP исключительно к `10.0.2.2` для AVD и `127.0.0.1` для USB с `adb reverse`. Release запрещает `cleartext` и до развертывания реального HTTPS использует нерабочий адрес `hotel.invalid`. Версия `0.6.0` является учебным локальным проектом, а не опубликованным коммерческим сервисом.

Выводы по разделу

Выбранная архитектура обеспечивает реализованный гостиничный сценарий от SQL-поиска до сохраненного заказа и нефискального подтверждения демооплаты. Авторитетными остаются серверные цена, права и время; транзакции, единый порядок блокировок и идемпотентность предотвращают частичные заказы и дубли. Снимки реквизитов сохраняют историю независимо от каталогных изменений.

Следующими работами являются Maps/Room, полноценный кабинет администратора и приемка на трех разных AVD. Они не объявляются выполненными в данной версии. Билеты, такси и автомобильная аренда требуют отдельного расширения требований, данных и API. Тем самым

документ разграничивает фактически реализованные этапы 3-6 и оставшуюся часть индивидуального задания.

Источники архитектурных решений

1. Индивидуальное задание: Арзимуротов А.М., ПИН-123. Мобильное приложение «Гостиница». 18.09.2026; срок сдачи 11.12.2026.

2. Android Developers. Guide to app architecture.

URL: <https://developer.android.com/topic/architecture>

Дата обращения: 02.10.2026.

3. PostgreSQL Documentation. Explicit Locking.

URL: <https://www.postgresql.org/docs/18/explicit-locking.html>

Дата обращения: 02.10.2026.

4. Apache PDFBox. Download and Documentation.

URL: <https://pdfbox.apache.org/download.html>

Дата обращения: 02.10.2026.

5. OWASP. Password Storage Cheat Sheet.

URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Дата обращения: 02.10.2026.